Eigenvalue Problems: Power Methods and the QR Algorithm

Lecture notes for MATH 616, Computational Linear Algebra
Dr. Xuping Xie, Old Dominion University

Fall 2026

Contents

Part A: Power methods **3**

1 Why eigenvalues need iteration **3**

1.1 Basic facts . . . 3

1.2 Where the eigenvalues are . . . 3

1.3 Eigenvalues are roots, and roots need iteration . . . 4

2 The power method **4**

2.1 The idea . . . 4

2.2 The algorithm . . . 5

2.3 Convergence of the scaled iteration . . . 7

2.4 The Rayleigh quotient . . . 7

3 The inverse and shifted inverse power method **8**

4 The power method for symmetric matrices **9**

5 Rayleigh quotient iteration **10**

6 Wielandt deflation **11**

Part B: The QR algorithm **12**

7 What the QR algorithm computes: the Schur form **12**

7.1 Decoupling . . . 12

7.2 Schur and real Schur form . . . 13

8 Householder reflectors and Givens rotations 13

8.1 Householder reflectors . . . 13

8.2 Givens rotations . . . 14

9 Reduction to Hessenberg and tridiagonal form 15

10 The basic QR algorithm 17

10.1 The iteration . . . 17

10.2 QR factorization of a tridiagonal matrix by Givens rotations . . . 17

10.3 How fast it converges, and when it does not . . . 19

11 Simultaneous iteration and why QR works 20

11.1 Gram–Schmidt is QR . . . 20

11.2 Simultaneous power iteration . . . 20

11.3 Simultaneous iteration is the QR algorithm . . . 21

12 The QR algorithm with shifts 22

12.1 The algorithm . . . 22

12.2 Choosing the shift . . . 22

12.3 Deflation in practice . . . 23

12.4 Computing the eigenvectors . . . 25

12.5 Summary . . . 25

13 Practice problems 25

Solutions 26

References 26

Part A: Power methods

1 Why eigenvalues need iteration

1.1 Basic facts

An eigenvalue of $A \in \mathbb{C}^{n\times n}$ is a scalar λ with $\det(A-\lambda I)=0$, and an associated eigenvector is a vector $v \neq 0$ with $Av=\lambda v$. The characteristic polynomial $p(\lambda)=\det(A-\lambda I)$ has degree n, so A has exactly n eigenvalues counted with multiplicity, possibly complex. We collect the facts used below; all of them were covered earlier in the course.

Theorem 1.1. *Let $A \in \mathbb{C}^{n\times n}$ have eigenvalues $\lambda_1, \dots, \lambda_n$.*

1. $\det A = \prod_{i=1}^{n} \lambda_i = p(0)$ *and* $\operatorname{tr} A = \sum_{i=1}^{n} a_{ii} = \sum_{i=1}^{n} \lambda_i$.
2. *Similar matrices A and $B = P^{-1}AP$ have the same eigenvalues.*
3. *Eigenvectors belonging to distinct eigenvalues are linearly independent.*
4. *If A is real symmetric, its eigenvalues are real and it has an orthonormal basis of eigenvectors. If it is also positive definite, every eigenvalue is positive.*
5. *If (λ, v) is an eigenpair of A, then $(\lambda-\alpha, v)$ is an eigenpair of $A-\alpha I$, and, when A is nonsingular, $(1/\lambda, v)$ is an eigenpair of A^{-1}.*

The last item is the engine of the shifted and inverse methods: shifting and inverting change the eigenvalues in a known way and leave the eigenvectors alone.

Example 1.2. For

$$A = \begin{pmatrix} 1 & 2 & 1 \\ 0 & 1 & 3 \\ 2 & 1 & 1 \end{pmatrix}, \qquad p(\lambda) = -\lambda^3 + 3\lambda^2 + 2\lambda + 8 = -(\lambda-4)(\lambda^2+\lambda+2),$$

so the eigenvalues are 4 and $(-1 \pm i\sqrt{7})/2$. Solving $(A-4I)v=0$ by elimination gives $v=(1,1,1)^{\mathsf{T}}$. The eigenvalues of A^{-1} are $1/4$ and $2/(-1\pm i\sqrt{7})$, with the same eigenvectors.

1.2 Where the eigenvalues are

Before computing eigenvalues it helps to know roughly where they lie.

Theorem 1.3 (Gershgorin)**.** *Let D_i be the closed disk in $\mathbb{C}$ with center a_{ii} and radius $r_i = \sum_{j\neq i} |a_{ij}|$. Every eigenvalue of A lies in $\bigcup_{i=1}^{n} D_i$. If the union of k of the disks is disjoint from the other $n-k$ disks, it contains exactly k eigenvalues.*

Proof of the first statement. Let $Av=\lambda v$ and choose i with $|v_i| = \|v\|_\infty > 0$. Row i of $Av=\lambda v$ gives $(\lambda - a_{ii})v_i = \sum_{j\neq i} a_{ij}v_j$, so $|\lambda - a_{ii}|\,|v_i| \le \sum_{j\neq i}|a_{ij}|\,|v_j| \le r_i|v_i|$. Divide by $|v_i|$. $\square$

Theorem 1.4. *For every subordinate matrix norm, every eigenvalue satisfies* $|\lambda| \le \rho(A) \le \|A\|$.

Example 1.5. For the symmetric matrix

$$A = \begin{pmatrix} 3 & 1 & 2 \\ 1 & -6 & 1 \\ 2 & 1 & 0 \end{pmatrix}$$

we have $\|A\|_\infty = 8$, and the eigenvalues are real, so they lie in $[-8, 8]$. The Gershgorin disks are the intervals $D_1 = [0, 6]$, $D_2 = [-8, -4]$, $D_3 = [-3, 3]$. Since D_2 is disjoint from $D_1 \cup D_3 = [-3, 6]$, there is exactly one eigenvalue in $[-8, -4]$ and two in $[-3, 6]$. Indeed the eigenvalues are -6.2146, -0.9630, 4.1775 (to four decimals). This matrix is used again below.

1.3 Eigenvalues are roots, and roots need iteration

Every polynomial is the characteristic polynomial of some matrix. The monic polynomial

$$p(\lambda) = \lambda^m + a_{m-1}\lambda^{m-1} + \cdots + a_1\lambda + a_0$$

is, up to sign, the characteristic polynomial of its *companion matrix*

$$C = \begin{pmatrix} 0 & 0 & \cdots & 0 & -a_0 \\ 1 & 0 & \cdots & 0 & -a_1 \\ 0 & 1 & \cdots & 0 & -a_2 \\ \vdots & & \ddots & & \vdots \\ 0 & 0 & \cdots & 1 & -a_{m-1} \end{pmatrix}.$$

So computing eigenvalues is at least as hard as finding polynomial roots. By the Abel–Ruffini theorem there is no formula in radicals for the roots of a general polynomial of degree five or more. Consequently *every* eigenvalue algorithm for general matrices of size $n \ge 5$ must be iterative: it produces a sequence of numbers that converges to the eigenvalues, and the only questions are how fast and at what cost per step.

The reverse direction is used in practice. The NumPy function `numpy.roots` computes polynomial roots as the eigenvalues of the companion matrix.

2 The power method

2.1 The idea

Multiply a vector by A over and over. Each multiplication stretches the component along an eigenvector v_i by λ_i, so the component belonging to the eigenvalue of largest modulus eventually dominates.

Theorem 2.1. *Let* $A \in \mathbb{C}^{n \times n}$ *have eigenvalues ordered so that*

$$|\lambda_1| > |\lambda_2| \ge |\lambda_3| \ge \ldots \ge |\lambda_n|,$$

with n linearly independent eigenvectors $Av_i = \lambda_i v_i$. Write $x^{(0)} = \sum_{i=1}^{n} \alpha_i v_i$ with $\alpha_1 \neq 0$. Then

$$\lim_{k\to\infty} \frac{A^k x^{(0)}}{\lambda_1^k} = \alpha_1 v_1, \qquad \frac{A^k x^{(0)}}{\lambda_1^k} = \alpha_1 v_1 + O(|\lambda_2/\lambda_1|^k)\,.$$

If in addition $\alpha_1\,(x^{(0)})^{\mathsf{H}} v_1 \neq 0$, then

$$\lim_{k\to\infty} \frac{\langle x^{(0)}, A^k x^{(0)}\rangle}{\langle x^{(0)}, A^{k-1} x^{(0)}\rangle} = \lambda_1.$$

Proof. Since $A^k v_i = \lambda_i^k v_i$,

$$A^k x^{(0)} = \sum_{i=1}^{n} \alpha_i \lambda_i^k v_i = \lambda_1^k \Big(\alpha_1 v_1 + \sum_{i=2}^{n} \alpha_i \Big(\frac{\lambda_i}{\lambda_1}\Big)^k v_i \Big).$$

Each ratio satisfies $|\lambda_i/\lambda_1| \le |\lambda_2/\lambda_1| < 1$, which proves the first two claims. For the third, let $\eta_i = \langle x^{(0)}, v_i\rangle = (x^{(0)})^{\mathsf{H}} v_i$. Then $\langle x^{(0)}, A^k x^{(0)}\rangle = \sum_i \alpha_i \eta_i \lambda_i^k$, and

$$\frac{\sum_i \alpha_i \eta_i \lambda_i^k}{\sum_i \alpha_i \eta_i \lambda_i^{k-1}} = \lambda_1 \, \frac{\alpha_1\eta_1 + O(|\lambda_2/\lambda_1|^k)}{\alpha_1\eta_1 + O(|\lambda_2/\lambda_1|^{k-1})} \longrightarrow \lambda_1. \qquad \square$$

The condition $\alpha_1 \neq 0$ says that the starting vector has some component along v_1. A random starting vector satisfies it with probability one, and in floating point arithmetic rounding errors usually introduce such a component even when it is missing.

2.2 The algorithm

The vectors $A^k x^{(0)}$ grow or shrink like $|\lambda_1|^k$, which soon overflows or underflows. The algorithm therefore rescales at every step, here so that the largest entry equals one.

1. Choose $x^{(0)}$ with $\|x^{(0)}\|_\infty = 1$, a tolerance, and a maximum number of iterations.
2. For $k = 1, 2, \ldots$:
   (a) $y^{(k)} = A x^{(k-1)}$;
   (b) choose the index p with $|y_p^{(k)}| = \|y^{(k)}\|_\infty$ and set $\beta^{(k)} = y_p^{(k)}$;
   (c) $x^{(k)} = y^{(k)} / \beta^{(k)}$;
   (d) stop when $|\beta^{(k)} - \beta^{(k-1)}|$ is below the tolerance.

```
import numpy as np

def power_method(A, x0, tol=1e-10, maxit=500):
    """Power method with infinity-norm scaling.

    Returns the eigenvalue estimate beta, the scaled eigenvector x
```

```
    (largest entry equal to 1), and the number of iterations used.
    """
    x = x0 / x0[np.argmax(np.abs(x0))]
    beta_old = 0.0
    for k in range(1, maxit + 1):
        y = A @ x
        p = np.argmax(np.abs(y))       # index of the largest entry
        beta = y[p]
        x = y / beta
        if abs(beta - beta_old) < tol:
            return beta, x, k
        beta_old = beta
    raise RuntimeError("power method did not converge")
```

Each step costs one matrix–vector product, $O(n^2)$ operations for a dense matrix and $O(\mathrm{nnz}(A))$ for a sparse one. The matrix itself is never modified.

Example 2.2. Take

$$A = \begin{pmatrix} 5 & 1 & 0 \\ 0 & 2 & 2 \\ 1 & 2 & -3 \end{pmatrix}, \qquad x^{(0)} = \left(-1, \tfrac{1}{2}, \tfrac{1}{2}\right)^{\mathsf{T}}.$$

The first two steps, in exact arithmetic, are

$$\begin{aligned} Ax^{(0)} &= \left(-\tfrac{9}{2},\, 2,\, -\tfrac{3}{2}\right)^{\mathsf{T}}, & \beta^{(1)} &= -\tfrac{9}{2}, & x^{(1)} &= \left(1,\, -\tfrac{4}{9},\, \tfrac{1}{3}\right)^{\mathsf{T}}, \\ Ax^{(1)} &= \left(\tfrac{41}{9},\, -\tfrac{2}{9},\, -\tfrac{8}{9}\right)^{\mathsf{T}}, & \beta^{(2)} &= \tfrac{41}{9}, & x^{(2)} &= \left(1,\, -\tfrac{2}{41},\, -\tfrac{8}{41}\right)^{\mathsf{T}}. \end{aligned}$$

The eigenvalues of A are approximately 5.0950, -3.6653 and 2.5703, so the convergence factor is $|\lambda_2/\lambda_1| \approx 0.719$. The estimates approach λ_1 slowly:

k	$\beta^{(k)}$	$\lvert\beta^{(k)} - \lambda_1\rvert$
1	-4.5000	9.5950
2	4.5556	0.5394
3	4.9512	0.1438
4	4.9015	0.1935
5	5.0824	0.0126
6	5.0245	0.0705
7	5.1057	0.0107
8	5.0668	0.0282

With tolerance 10^{-10} the code stops after 69 iterations with $\beta = 5.0949940$ and

$$x = (1,\ 0.094994,\ 0.147003)^{\mathsf{T}}.$$

The count matches the theory: $0.719^{69} \approx 10^{-10}$.

2.3 Convergence of the scaled iteration

Theorem 2.3. *Under the hypotheses of Theorem 2.1, the iterates of the power method satisfy*

$$\lim_{k\to\infty} x^{(k)} = \frac{v_1}{v_{1,p}}, \qquad \|x^{(k)} - a_k v_1\| = O(|\lambda_2/\lambda_1|^k) \text{ for suitable scalars } a_k,$$

where $v_{1,p}$ is an entry of v_1 of largest modulus, and

$$\lim_{k\to\infty} \beta^{(k)} = \lambda_1, \qquad |\beta^{(k)} - \lambda_1| = O(|\lambda_2/\lambda_1|^k)\,.$$

Proof. By induction $x^{(k)} = c_k A^k x^{(0)}$ with $c_k = 1/(\beta^{(1)}\beta^{(2)}\cdots\beta^{(k)})$, so

$$x^{(k)} = c_k \lambda_1^k \Big(\alpha_1 v_1 + \sum_{i=2}^{n} \alpha_i (\lambda_i/\lambda_1)^k v_i \Big).$$

The left side has infinity norm one and the bracket tends to $\alpha_1 v_1$, so $|c_k\lambda_1^k| \to 1/(|\alpha_1|\|v_1\|_\infty)$. With $a_k = c_k\lambda_1^k\alpha_1$ this gives $x^{(k)} - a_k v_1 = O(|\lambda_2/\lambda_1|^k)$. Since the largest entry of $x^{(k)}$ equals one, its limit is v_1 scaled so that its largest entry is one. For the eigenvalue, $\beta^{(k)} = (Ax^{(k-1)})_p = y_p^{(k)}$, and for k large the index p is the position of the largest entry of v_1. Using the expansion of $A^k x^{(0)}$ in both numerator and denominator,

$$\beta^{(k)} = \frac{(A^k x^{(0)})_p}{(A^{k-1}x^{(0)})_p} = \lambda_1 \frac{\alpha_1 v_{1,p} + O(|\lambda_2/\lambda_1|^k)}{\alpha_1 v_{1,p} + O(|\lambda_2/\lambda_1|^{k-1})} = \lambda_1\big(1 + O(|\lambda_2/\lambda_1|^{k-1})\big). \qquad \square$$

Remark 2.4. Both the eigenvalue and the eigenvector converge *linearly*, with factor $|\lambda_2/\lambda_1|$. When this factor is close to one, as in Example 2.2, the method is slow. Because the error behaves like Cr^k, Aitken's Δ^2 extrapolation applied to the sequence $\beta^{(k)}$ can accelerate it. The more powerful remedy is to change the matrix, which is the subject of the next section.

2.4 The Rayleigh quotient

From $\beta^{(k)}x^{(k)} = Ax^{(k-1)}$ and $x^{(k-1)} \to v_1$ one sees that $\beta^{(k)}$ tends to $v_1^{\mathsf{H}}Av_1/v_1^{\mathsf{H}}v_1 = \lambda_1$. This suggests a better eigenvalue estimate from an approximate eigenvector.

Definition 2.5. The *Rayleigh quotient* of a nonzero vector x with respect to A is

$$r(x) = \frac{\langle x, Ax\rangle}{\langle x, x\rangle} = \frac{x^{\mathsf{H}}Ax}{x^{\mathsf{H}}x}.$$

If x is an eigenvector with eigenvalue λ, then $r(x) = \lambda$. For real symmetric A the gradient of r vanishes at every eigenvector, so an eigenvector error of size ε produces an eigenvalue error of size $O(\varepsilon^2)$. This quadratic accuracy is what the symmetric power method and Rayleigh quotient iteration exploit.

3 The inverse and shifted inverse power method

The power method finds only the eigenvalue of largest modulus. To reach the others, apply it to a different matrix with the same eigenvectors.

Let α be a number that is not an eigenvalue of A. By Theorem 1.1, the matrix $(A-\alpha I)^{-1}$ has eigenvalues

$$\mu_i = \frac{1}{\lambda_i - \alpha},$$

with the same eigenvectors as A. The largest $|\mu_i|$ belongs to the eigenvalue λ_i *closest to* α. The power method applied to $(A-\alpha I)^{-1}$ therefore converges to that μ, and the eigenvalue of A is recovered as $\lambda = \alpha + 1/\mu$. By choosing α near any eigenvalue of interest, for instance from Gershgorin disks, one can compute any eigenvalue.

The inverse is never formed. Each step solves a linear system with the fixed matrix $A-\alpha I$, so this matrix is factored once at cost $O(n^3)$ and each step then costs two triangular solves, $O(n^2)$.

1. Choose α and $x^{(0)}$ with $\|x^{(0)}\|_\infty = 1$. Factor $A - \alpha I = PLU$.
2. For $k = 1, 2, \ldots$: solve $(A-\alpha I)y^{(k)} = x^{(k-1)}$; set $\mu^{(k)} = y^{(k)}_p$ with $|y^{(k)}_p| = \|y^{(k)}\|_\infty$; set $x^{(k)} = y^{(k)}/\mu^{(k)}$.
3. Return $\lambda \approx \alpha + 1/\mu^{(k)}$ and $x^{(k)}$.

```
from scipy.linalg import lu_factor, lu_solve

def inverse_power(A, alpha, x0, tol=1e-10, maxit=500):
    """Shifted inverse power method: eigenvalue of A closest to alpha."""
    n = A.shape[0]
    lu = lu_factor(A - alpha * np.eye(n))   # factor once, O(n^3)
    x = x0 / x0[np.argmax(np.abs(x0))]
    mu_old = 0.0
    for k in range(1, maxit + 1):
        y = lu_solve(lu, x)                  # each solve is O(n^2)
        p = np.argmax(np.abs(y))
        mu = y[p]
        x = y / mu
        if abs(mu - mu_old) < tol * abs(mu):
            return alpha + 1.0 / mu, x, k
        mu_old = mu
    raise RuntimeError("inverse power method did not converge")
```

Theorem 3.1. *Order the eigenvalues by distance from the shift, $|\lambda_1-\alpha| < |\lambda_2-\alpha| \le \ldots \le |\lambda_n-\alpha|$, and assume a basis of eigenvectors. Then*

$$\left\| x^{(k)} - \frac{v_1}{v_{1,p}} \right\| = O\left(\left| \frac{\lambda_1-\alpha}{\lambda_2-\alpha} \right|^k \right), \qquad \left| \mu^{(k)} - \frac{1}{\lambda_1-\alpha} \right| = O\left(\left| \frac{\lambda_1-\alpha}{\lambda_2-\alpha} \right|^k \right).$$

Proof. This is Theorem 2.3 applied to $(A-\alpha I)^{-1}$, whose two largest eigenvalues in modulus are $1/(\lambda_1-\alpha)$ and $1/(\lambda_2-\alpha)$. $\square$

The closer the shift is to λ_1, the smaller the ratio and the faster the convergence. One might worry that $A-\alpha I$ is then nearly singular and the solves are inaccurate. The error in $y^{(k)}$ is indeed large, but it points almost entirely along v_1, the direction we want, so the normalized iterate is still accurate.

Example 3.2. For the matrix of Example 2.2, starting from $x^{(0)}=(1,1,1)^{\mathsf{T}}$ with tolerance 10^{-10}:

Shift α	Eigenvalue found	Ratio $\lvert(\lambda_1-\alpha)/(\lambda_2-\alpha)\rvert$	Iterations
2	2.5703171	0.184	15
-3	-3.6653111	0.215	13
5	5.0949940	0.039	9

The shift $\alpha=5$ finds the dominant eigenvalue in 9 iterations instead of the 69 needed by the plain power method.

4 The power method for symmetric matrices

When A is real symmetric, its eigenvectors are orthogonal, and using the Rayleigh quotient as the eigenvalue estimate doubles the rate of convergence.

1. Choose $x^{(0)}$ with $\|x^{(0)}\|_2=1$.
2. For $k=1,2,\ldots$: $y^{(k)}=Ax^{(k-1)}$; $\beta^{(k)}=(x^{(k-1)})^{\mathsf{T}}y^{(k)}$; $x^{(k)}=y^{(k)}/\|y^{(k)}\|_2$.

Here $\beta^{(k)}=(x^{(k-1)})^{\mathsf{T}}Ax^{(k-1)}=r(x^{(k-1)})$ is exactly the Rayleigh quotient of the current unit vector.

```
def symmetric_power(A, x0, tol=1e-10, maxit=500):
    """Power method for symmetric A, 2-norm scaling, Rayleigh quotient."""
    x = x0 / np.linalg.norm(x0)
    beta_old = 0.0
    for k in range(1, maxit + 1):
        y = A @ x
        beta = x @ y                          # Rayleigh quotient of x
        x = y / np.linalg.norm(y)
        if abs(beta - beta_old) < tol * abs(beta):
            return beta, x, k
        beta_old = beta
    raise RuntimeError("symmetric power method did not converge")
```

Theorem 4.1. *Let A be real symmetric with orthonormal eigenvectors v_i and eigenvalues $|\lambda_1|>|\lambda_2|\ge\ldots\ge|\lambda_n|$, and let $x^{(0)}=\sum_i \alpha_i v_i$ with $\alpha_1\neq 0$. Then $x^{(k)}\mp v_1=O(|\lambda_2/\lambda_1|^{k})$ for a suitable choice of sign, and*

$$|\beta^{(k)}-\lambda_1|=O(|\lambda_2/\lambda_1|^{2k})\,.$$

Proof. As before $x^{(k-1)} = c\,A^{k-1}x^{(0)} = c\,\lambda_1^{k-1}\sum_i \alpha_i (\lambda_i/\lambda_1)^{k-1} v_i$ for some scalar c, which gives the eigenvector statement. The Rayleigh quotient is independent of the scaling, and orthonormality $\langle v_i, v_j\rangle = \delta_{ij}$ removes all cross terms:

$$\beta^{(k)} = \frac{\langle x^{(k-1)}, A x^{(k-1)}\rangle}{\langle x^{(k-1)}, x^{(k-1)}\rangle} = \lambda_1 \,\frac{\alpha_1^2 + \sum_{i\ge 2} \alpha_i^2 (\lambda_i/\lambda_1)^{2k-1}}{\alpha_1^2 + \sum_{i\ge 2} \alpha_i^2 (\lambda_i/\lambda_1)^{2k-2}}.$$

Both sums are $O(|\lambda_2/\lambda_1|^{2k-2})$, so $\beta^{(k)} = \lambda_1(1 + O(|\lambda_2/\lambda_1|^{2k-2}))$. □

The rate is still linear, but with factor $|\lambda_2/\lambda_1|^2$, so the symmetric method needs about half as many iterations as the basic method for the same accuracy in the eigenvalue. The eigenvector still converges with factor $|\lambda_2/\lambda_1|$.

Example 4.2. For the symmetric matrix of the Gershgorin example, the dominant eigenvalue is -6.2145729 and $|\lambda_2/\lambda_1| = 4.1775/6.2146 \approx 0.672$. Starting from $(1,1,1)^{\mathsf T}$ with relative tolerance 10^{-10}, the symmetric power method stops after 33 iterations, while the basic power method needs 64.

5 Rayleigh quotient iteration

The inverse power method converges quickly when the shift is close to an eigenvalue, and the Rayleigh quotient of the current iterate is an excellent estimate of that eigenvalue. Rayleigh quotient iteration combines the two by updating the shift at every step.

1. Choose $x^{(0)}$ with $\|x^{(0)}\|_2 = 1$ and set $\beta^{(0)} = (x^{(0)})^{\mathsf T} A x^{(0)}$.
2. For $k = 1, 2, \ldots$:
   (a) solve $(A - \beta^{(k-1)} I)\, y^{(k)} = x^{(k-1)}$ (one inverse iteration step);
   (b) normalize, $x^{(k)} = y^{(k)} / \|y^{(k)}\|_2$;
   (c) update the shift, $\beta^{(k)} = (x^{(k)})^{\mathsf T} A x^{(k)}$.

```
def rayleigh_quotient_iteration(A, x0, tol=1e-12, maxit=50):
    """Rayleigh quotient iteration: inverse iteration with shift r(x)."""
    n = A.shape[0]
    x = x0 / np.linalg.norm(x0)
    beta = x @ A @ x
    for k in range(1, maxit + 1):
        try:
            y = np.linalg.solve(A - beta * np.eye(n), x)
        except np.linalg.LinAlgError:        # shift is an exact eigenvalue
            return beta, x, k
        x = y / np.linalg.norm(y)
        beta_new = x @ A @ x
        if np.linalg.norm(A @ x - beta_new * x) < tol:
            return beta_new, x, k
```

```
        beta = beta_new
    raise RuntimeError("Rayleigh quotient iteration did not converge")
```

Theorem 5.1 (Trefethen and Bau, Lecture 27)**.** *Let A be real symmetric. Rayleigh quotient iteration converges to an eigenpair for all starting vectors except a set of measure zero. When it converges to (λ_J, v_J), the convergence is cubic: for $x^{(k)}$ sufficiently close to $\pm v_J$,*

$$\|x^{(k+1)} - (\pm v_J)\| = O\Big(\|x^{(k)} - (\pm v_J)\|^3\Big), \qquad |\beta^{(k+1)} - \lambda_J| = O\Big(|\beta^{(k)} - \lambda_J|^3\Big).$$

The cubic rate comes from two quadratic effects working together. An eigenvector error ε gives a shift error $O(\varepsilon^2)$ (the Rayleigh quotient is quadratically accurate), and one inverse iteration step with a shift that close reduces the eigenvector error by a factor proportional to the shift error, giving $O(\varepsilon \cdot \varepsilon^2)$. For nonsymmetric matrices the Rayleigh quotient is only linearly accurate and the rate drops to quadratic.

Example 5.2. For the symmetric 3×3 matrix of the Gershgorin example, starting from $x^{(0)} = (1,1,1)^{\mathsf{T}}/\sqrt{3}$, the method converges to $\lambda = 4.1775340$, which is not the dominant eigenvalue; Rayleigh quotient iteration converges to whichever eigenvalue the starting vector favors. The residuals $\|Ax^{(k)} - \beta^{(k)}x^{(k)}\|_2$ show the cubic rate, with the number of correct digits roughly tripling at each step:

k	$\beta^{(k)}$	Residual
1	3.4397150573777	2.1×10^{0}
2	4.1606665361667	3.1×10^{-1}
3	4.1775337860050	9.5×10^{-4}
4	4.1775339552426	3.0×10^{-11}
5	4.1775339552426	6.4×10^{-16}

Remark 5.3. The price of the changing shift is that $A - \beta^{(k)}I$ must be factored afresh at every step, $O(n^3)$ for a dense matrix. For symmetric A one first reduces A to a similar tridiagonal matrix with Householder transformations (Section 9). A tridiagonal system can be solved in $O(n)$ operations, so each step of Rayleigh quotient iteration then costs only $O(n)$.

6 Wielandt deflation

Once λ_1 and v_1 are known, deflation builds a matrix that has the remaining eigenvalues, so that the power method can be applied again to find λ_2.

Assume $|\lambda_1| > |\lambda_2| > |\lambda_3| \geq \ldots \geq |\lambda_n|$, ordered by modulus. Compute (λ_1, v_1) with the power method. Let i be an index with $|v_{1,i}| = \|v_1\|_\infty$, and define

$$x = \frac{1}{\lambda_1 v_{1,i}}\big(a_{i1}, a_{i2}, \ldots, a_{in}\big)^{\mathsf{T}}, \qquad B = A - \lambda_1 v_1 x^{\mathsf{T}}.$$

Proposition 6.1. *The matrix B has eigenvalues $0, \lambda_2, \ldots, \lambda_n$. Moreover, row i of B is identically zero.*

Proof. First, $x^{\mathsf{T}} v_1 = (Av_1)_i/(\lambda_1 v_{1,i}) = \lambda_1 v_{1,i}/(\lambda_1 v_{1,i}) = 1$, so $Bv_1 = \lambda_1 v_1 - \lambda_1 v_1 = 0$, and 0 is an eigenvalue. Next, use the transpose, $B^{\mathsf{T}} = A^{\mathsf{T}} - \lambda_1 x v_1^{\mathsf{T}}$, and any left eigenvector u_j of A for $\lambda_j \neq \lambda_1$ satisfies $v_1^{\mathsf{T}} u_j = 0$ (left and right eigenvectors of distinct eigenvalues are orthogonal). Hence $B^{\mathsf{T}} u_j = A^{\mathsf{T}} u_j = \lambda_j u_j$, so each λ_j, $j \geq 2$, is an eigenvalue of B^{T} and therefore of B. Finally, row i of B is $a_{i,:} - \lambda_1 v_{1,i}\, x^{\mathsf{T}} = a_{i,:} - a_{i,:} = 0$. □

Since the eigenvalue of largest modulus of B is now λ_2, the power method applied to B converges to λ_2. Because row i of B is zero, expanding $\det(B - \lambda I)$ along that row gives $\det(B - \lambda I) = -\lambda \det(B' - \lambda I)$, where B' is B with row i and column i deleted. So in practice one deletes row and column i and continues with the $(n-1) \times (n-1)$ matrix B', whose eigenvalues are exactly $\lambda_2, \ldots, \lambda_n$. Repeating the process yields all the eigenvalues, one at a time.

```
def wielandt_deflate(A, lam1, v1):
    """Wielandt deflation: B has eigenvalues 0, lambda_2, ..., lambda_n."""
    i = np.argmax(np.abs(v1))
    x = A[i, :] / (lam1 * v1[i])            # chosen so that x @ v1 = 1
    return A - lam1 * np.outer(v1, x)
```

Example 6.2. For the matrix of Example 2.2, deflating with the computed pair $(\lambda_1, v_1) \approx (5.0950,\, (1, 0.0950, 0.1470)^{\mathsf{T}})$ gives a matrix B with eigenvalues -3.6653, 2.5703 and 0 and a zero first row. The power method applied to B returns $\lambda_2 = -3.6653111$.

Remark 6.3. Each deflation uses a computed, hence inexact, eigenpair, so errors accumulate from one eigenvalue to the next. Deflation is useful for a few eigenvalues; to compute all of them one uses the QR algorithm of Part B, which performs a numerically stable version of deflation automatically.

Part B: The QR algorithm

7 What the QR algorithm computes: the Schur form

7.1 Decoupling

Most eigenvalue algorithms reduce a problem to smaller ones as soon as possible, using the following lemma.

Lemma 7.1. *If*

$$A = \begin{pmatrix} A_{11} & A_{12} \\ 0 & A_{22} \end{pmatrix}$$

with square diagonal blocks, then $\sigma(A) = \sigma(A_{11}) \cup \sigma(A_{22})$*, counting multiplicities.*

Proof. $\det(A - \lambda I) = \det(A_{11} - \lambda I)\det(A_{22} - \lambda I)$, since the determinant of a block upper triangular matrix is the product of the determinants of its diagonal blocks. □

7.2 Schur and real Schur form

A matrix $Q \in \mathbb{C}^{n\times n}$ is *unitary* if $Q^{\mathsf{H}}Q = QQ^{\mathsf{H}} = I$ (orthogonal, when real), and A is *normal* if $A^{\mathsf{H}}A = AA^{\mathsf{H}}$.

Theorem 7.2 (Schur)**.** *For every $A \in \mathbb{C}^{n\times n}$ there is a unitary Q such that $Q^{\mathsf{H}}AQ = T$ is upper triangular, with the eigenvalues of A on the diagonal of T in any prescribed order.*

Lemma 7.3. *An upper triangular matrix that is normal is diagonal.*

Proof. Compare the $(1,1)$ entries of $T^{\mathsf{H}}T$ and TT^{H}: $|t_{11}|^2 = \sum_j |t_{1j}|^2$, so $t_{1j} = 0$ for $j > 1$. Repeat with the trailing submatrix. □

Corollary 7.4. *A normal matrix is unitarily diagonalizable: $Q^{\mathsf{H}}AQ = \operatorname{diag}(\lambda_1, \dots, \lambda_n)$. In particular this holds for real symmetric matrices, with Q real orthogonal.*

Proof. If $Q^{\mathsf{H}}AQ = T$ with A normal, then T is normal and upper triangular, hence diagonal. □

A real matrix may have complex eigenvalues, and software prefers to work in real arithmetic. The real analogue of the Schur form allows 2×2 diagonal blocks.

Theorem 7.5 (Real Schur form)**.** *For every $A \in \mathbb{R}^{n\times n}$ there is a real orthogonal Q such that*

$$Q^{\mathsf{T}}AQ = R = \begin{pmatrix} R_{11} & R_{12} & \cdots & R_{1m} \\ 0 & R_{22} & \cdots & R_{2m} \\ \vdots & & \ddots & \vdots \\ 0 & 0 & \cdots & R_{mm} \end{pmatrix},$$

where each R_{ii} is either 1×1 (a real eigenvalue) or 2×2 with a pair of complex conjugate eigenvalues.

The QR algorithm computes exactly this: in real arithmetic it converges to a real Schur form, and for symmetric matrices to the diagonal form $Q^{\mathsf{T}}AQ = \Lambda$. Its building blocks are orthogonal transformations, which do not amplify rounding errors. The two we need are Householder reflectors and Givens rotations.

8 Householder reflectors and Givens rotations

8.1 Householder reflectors

Definition 8.1. For $u \in \mathbb{R}^n$, $u \neq 0$, the *Householder matrix* is

$$H = I - 2\,\frac{uu^{\mathsf{T}}}{u^{\mathsf{T}}u}, \qquad \text{or} \quad H = I - 2uu^{\mathsf{T}} \text{ when } \|u\|_2 = 1.$$

Proposition 8.2. *H is symmetric, orthogonal, and an involution: $H^{\mathsf{T}} = H$ and $H^2 = H^{\mathsf{T}}H = I$.*

Proof. Symmetry is clear. With $\|u\|_2 = 1$, $H^2 = I - 4uu^{\mathsf{T}} + 4u(u^{\mathsf{T}}u)u^{\mathsf{T}} = I$. □

Geometrically, H reflects across the hyperplane orthogonal to u. Its main use is to zero all entries of a vector below the first.

Proposition 8.3. *Let $w \in \mathbb{R}^n$, $w \neq 0$, and set*

$$\alpha = -\operatorname{sign}(w_1)\,\|w\|_2 \quad (\textit{with } \operatorname{sign}(0) = 1), \qquad y = w - \alpha e_1, \qquad u = \frac{y}{\|y\|_2}.$$

Then $(I - 2uu^{\mathsf{T}})w = \alpha e_1$.

Proof. Because α has the sign opposite to w_1, we have $-\alpha w_1 = |w_1|\|w\|_2$. Hence

$$\|y\|_2^2 = \|w\|_2^2 - 2\alpha w_1 + \alpha^2 = 2\big(\|w\|_2^2 + |w_1|\|w\|_2\big), \qquad u^{\mathsf{T}}w = \frac{\|w\|_2^2 - \alpha w_1}{\|y\|_2} = \frac{\|w\|_2^2 + |w_1|\|w\|_2}{\|y\|_2}.$$

Therefore $2uu^{\mathsf{T}}w = 2\,\dfrac{\|w\|_2^2 + |w_1|\|w\|_2}{\|y\|_2^2}\,y = y = w - \alpha e_1$, and $Hw = w - (w - \alpha e_1) = \alpha e_1$. □

The sign choice matters numerically. With it, $y_1 = w_1 - \alpha = w_1 + \operatorname{sign}(w_1)\|w\|_2$ is a sum of two numbers of the same sign, so there is no cancellation, and $\|y\|_2$ is never small relative to $\|w\|_2$.

```
def house(w):
    """Unit vector u with (I - 2 u u^T) w = alpha e_1, alpha = -sign(w_0)||w||."""
    s = 1.0 if w[0] >= 0 else -1.0
    alpha = -s * np.linalg.norm(w)
    u = w.astype(float).copy()
    u[0] -= alpha
    return u / np.linalg.norm(u), alpha
```

A reflector is never formed as a matrix. To apply it, compute $HB = B - 2u(u^{\mathsf{T}}B)$, which costs $O(mn)$ for an $m \times n$ block instead of $O(m^2 n)$.

Householder QR factorization. Applying this construction to the columns of A in turn gives $A = QR$. In step k, take w to be the part of column k of the current matrix on and below the diagonal, build the reflector H_k that zeros the entries below the diagonal, and update $A^{(k)} = H_k A^{(k-1)}$. After $n - 1$ steps $H_{n-1} \cdots H_1 A = R$ is upper triangular, and $A = QR$ with $Q = H_1 H_2 \cdots H_{n-1}$.

8.2 Givens rotations

Definition 8.4. The *Givens rotation* $G(i, j, \theta)$ is the identity matrix except in the four entries

$$g_{ii} = c, \quad g_{ij} = s, \quad g_{ji} = -s, \quad g_{jj} = c, \qquad c = \cos\theta,\ s = \sin\theta.$$

Applied to x, it changes only entries i and j: $y_i = cx_i + sx_j$ and $y_j = -sx_i + cx_j$. Premultiplication by $G(i, j, \theta)$ rotates the (x_i, x_j) plane clockwise by the angle θ, and multiplication by G^{T} rotates it counterclockwise. To zero y_j choose

$$c = \frac{x_i}{\sqrt{x_i^2 + x_j^2}}, \qquad s = \frac{x_j}{\sqrt{x_i^2 + x_j^2}}.$$

The angle θ is never computed. To avoid overflow in $x_i^2 + x_j^2$, divide by the larger entry first:

```
def givens(a, b):
    """c, s with [c s; -s c] [a; b] = [r; 0], computed without overflow."""
    if b == 0:
        return 1.0, 0.0
    if abs(b) > abs(a):
        tau = a / b
        s = 1.0 / np.sqrt(1.0 + tau**2)
        c = s * tau
    else:
        tau = b / a
        c = 1.0 / np.sqrt(1.0 + tau**2)
        s = c * tau
    return c, s
```

Computing c and s takes about five flops and one square root. A rotation is orthogonal, $GG^{\mathsf{T}} = G^{\mathsf{T}}G = I$, and it modifies only two rows, so applying it to a matrix with n columns costs $6n$ flops.

When to use which. A Householder reflector zeros a whole column segment at once and is the tool of choice for dense matrices. A Givens rotation zeros one chosen entry and touches only two rows, so it is preferred when only a few entries must be eliminated, as in the QR factorization of a tridiagonal or Hessenberg matrix below. Givens rotations can also reduce a matrix to Hessenberg form, at a cost of about $4n^3/3$ multiplications, which is more than the Householder approach.

9 Reduction to Hessenberg and tridiagonal form

A matrix is *upper Hessenberg* if $a_{ij} = 0$ for $i > j + 1$, that is, it is zero below the first subdiagonal. A QR step on a full matrix costs $O(n^3)$, but on a Hessenberg matrix it costs only $O(n^2)$, and on a symmetric tridiagonal matrix only $O(n)$. The QR algorithm therefore begins with a one-time reduction to this form by orthogonal *similarity* transformations, which do not change the eigenvalues.

One cannot go all the way to triangular form in finitely many steps (that would contradict the Abel–Ruffini theorem). Hessenberg form is the closest one can get directly. The reflector in step k is designed to zero the entries of column k below the *subdiagonal*, and it does not touch row or column k or earlier ones when applied from the right, so the zeros survive.

Step k $(k = 1, \dots, n-2)$. Let $w = (0, \dots, 0, a_{k+1,k}, a_{k+2,k}, \dots, a_{nk})^{\mathsf T}$ (with k leading zeros), and set

$$\alpha = -\operatorname{sign}(a_{k+1,k}) \Big(\sum_{j=k+1}^{n} a_{jk}^2 \Big)^{1/2}, \qquad r = \Big(\tfrac{1}{2}\alpha^2 - \tfrac{1}{2} a_{k+1,k}\alpha \Big)^{1/2},$$

$$u^{(k)} = \frac{1}{2r}\big(0, \dots, 0,\ a_{k+1,k} - \alpha,\ a_{k+2,k}, \dots, a_{nk}\big)^{\mathsf T}, \qquad H^{(k)} = I - 2u^{(k)}u^{(k)\mathsf T}.$$

One checks that $\|u^{(k)}\|_2 = 1$, which is Proposition 8.3 applied to the trailing part of the column. Then $A^{(k+1)} = H^{(k)} A^{(k)} H^{(k)}$. After $n-2$ steps,

$$A^{(n-1)} = H^{(n-2)} \cdots H^{(1)}\, A\, H^{(1)} \cdots H^{(n-2)} = HAH^{\mathsf T}, \qquad H = H^{(n-2)} \cdots H^{(1)},$$

which is upper Hessenberg. If A is symmetric, then so is $HAH^{\mathsf T}$, and a symmetric Hessenberg matrix is *tridiagonal*.

Example 9.1. Reduce the symmetric matrix

$$A = \begin{pmatrix} 4 & 1 & -2 & 2 \\ 1 & 2 & 0 & 1 \\ -2 & 0 & 3 & -2 \\ 2 & 1 & -2 & -1 \end{pmatrix}.$$

Step 1. $\sum_{j=2}^{4} a_{j1}^2 = 9$, so $\alpha = -3$, $2r^2 = \alpha^2 - \alpha a_{21} = 12$, $r = \sqrt{6}$, and $u^{(1)} = \frac{1}{2\sqrt{6}}(0, 4, -2, 2)^{\mathsf T}$. Then

$$H^{(1)} = \begin{pmatrix} 1 & 0 & 0 & 0 \\ 0 & -1/3 & 2/3 & -2/3 \\ 0 & 2/3 & 2/3 & 1/3 \\ 0 & -2/3 & 1/3 & 2/3 \end{pmatrix}, \qquad A^{(2)} = H^{(1)} A H^{(1)} = \begin{pmatrix} 4 & -3 & 0 & 0 \\ -3 & 10/3 & 1 & 4/3 \\ 0 & 1 & 5/3 & -4/3 \\ 0 & 4/3 & -4/3 & -1 \end{pmatrix}.$$

Step 2. $a_{32}^2 + a_{42}^2 = 1 + 16/9 = 25/9$, so $\alpha = -5/3$, $2r^2 = \alpha^2 - \alpha a_{32} = 40/9$, and $u^{(2)} = \frac{1}{2r}(0, 0, 8/3, 4/3)^{\mathsf T}$. The trailing 2×2 block of $H^{(2)}$ is

$$\begin{pmatrix} -3/5 & -4/5 \\ -4/5 & 3/5 \end{pmatrix}, \qquad \text{and} \qquad A^{(3)} = H^{(2)} A^{(2)} H^{(2)} = \begin{pmatrix} 4 & -3 & 0 & 0 \\ -3 & 10/3 & -5/3 & 0 \\ 0 & -5/3 & -33/25 & 68/75 \\ 0 & 0 & 68/75 & 149/75 \end{pmatrix}.$$

$A^{(3)}$ is symmetric tridiagonal and similar to A. As a check, both have trace 8 and eigenvalues -2.1975, 1.0844, 2.2685, 6.8446.

An efficient update. For symmetric $A^{(k)}$ and unit $u = u^{(k)}$, expand the product:

$$HAH = A - 2up^{\mathsf T} - 2pu^{\mathsf T} + 4(u^{\mathsf T}p)\, uu^{\mathsf T}, \qquad p = Au.$$

With $q = p - (u^{\mathsf T}p)\, u$ this collapses to the symmetric rank-two update

$$A^{(k+1)} = A^{(k)} - 2\big(uq^{\mathsf T} + qu^{\mathsf T}\big),$$

which costs one matrix–vector product and one rank-two update. Only the trailing $(n-k) \times (n-k)$ block changes, so step k costs about $4(n-k)^2$ flops, and the whole reduction of a symmetric matrix costs about $\frac{4}{3}n^3$ flops (Golub and Van Loan; Trefethen and Bau). For a nonsymmetric matrix the reduction to Hessenberg form costs about $\frac{10}{3}n^3$ flops.

```
def hessenberg_reduce(A):
    """Reduce A to upper Hessenberg form H = Q^T A Q by Householder steps.

    For symmetric A the result is symmetric tridiagonal.
    """
    H = A.astype(float).copy()
    n = H.shape[0]
    Q = np.eye(n)
    for k in range(n - 2):
        if np.linalg.norm(H[k+2:, k]) == 0:
            continue                          # column already reduced
        u, _ = house(H[k+1:, k])
        H[k+1:, :] -= 2.0 * np.outer(u, u @ H[k+1:, :])   # from the left
        H[:, k+1:] -= 2.0 * np.outer(H[:, k+1:] @ u, u)   # from the right
        Q[:, k+1:] -= 2.0 * np.outer(Q[:, k+1:] @ u, u)
    return H, Q
```

Run on the 4×4 example, this returns the tridiagonal matrix $A^{(3)}$ above to machine precision, and $Q^{\mathsf{T}} A Q = H$. SciPy provides the same operation as `scipy.linalg.hessenberg`.

10 The basic QR algorithm

10.1 The iteration

Starting from $A^{(0)} = A$, factor and multiply in reverse order:

$$A^{(k-1)} = Q^{(k)} R^{(k)}, \qquad A^{(k)} = R^{(k)} Q^{(k)}, \qquad k = 1, 2, \ldots$$

Theorem 10.1. *Every iterate is orthogonally similar to its predecessor: $A^{(k)} = Q^{(k)\mathsf{T}} A^{(k-1)} Q^{(k)}$. Hence all iterates have the eigenvalues of A. If A is symmetric (or tridiagonal, or Hessenberg), so is every $A^{(k)}$. If A is symmetric with $|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n|$, then*

$$\lim_{k \to \infty} A^{(k)} = \operatorname{diag}(\lambda_1, \lambda_2, \ldots, \lambda_n).$$

Proof of similarity. $R^{(k)} = Q^{(k)\mathsf{T}} A^{(k-1)}$, so $A^{(k)} = R^{(k)} Q^{(k)} = Q^{(k)\mathsf{T}} A^{(k-1)} Q^{(k)}$. The convergence statement is proved in Section 11. $\square$

10.2 QR factorization of a tridiagonal matrix by Givens rotations

For a symmetric tridiagonal matrix with diagonal $a_1, \ldots, a_n$ and off-diagonal $b_2, \ldots, b_n$, only the $n - 1$ subdiagonal entries need to be zeroed, one Givens rotation each.

The first rotation acts on rows 1 and 2 with

$$c_1 = \frac{a_1}{D_1}, \quad s_1 = \frac{b_2}{D_1}, \quad D_1 = \sqrt{a_1^2 + b_2^2}.$$

It replaces rows 1 and 2 by $(z_1, q_1, r_1, 0, \dots)$ and $(0, x_2, y_2, 0, \dots)$ with

$$z_1 = D_1, \quad q_1 = \frac{a_1 b_2 + b_2 a_2}{D_1}, \quad r_1 = \frac{b_2 b_3}{D_1}, \quad x_2 = \frac{a_1 a_2 - b_2^2}{D_1}, \quad y_2 = \frac{a_1 b_3}{D_1}.$$

In general, before step k row k reads $(\dots, 0, x_k, y_k, 0, \dots)$ and row $k{+}1$ is still $(\dots, b_{k+1}, a_{k+1}, b_{k+2}, \dots)$. The rotation with

$$c = \frac{x_k}{D_k}, \quad s = \frac{b_{k+1}}{D_k}, \quad D_k = \sqrt{x_k^2 + b_{k+1}^2}$$

produces

$$z_k = D_k, \quad q_k = \frac{x_k y_k + b_{k+1} a_{k+1}}{D_k}, \quad r_k = \frac{b_{k+1} b_{k+2}}{D_k}, \quad x_{k+1} = \frac{x_k a_{k+1} - y_k b_{k+1}}{D_k}, \quad y_{k+1} = \frac{x_k b_{k+2}}{D_k}.$$

After $n-1$ rotations $G^{(n-1)} \cdots G^{(1)} A = R$ is upper triangular with only three nonzero diagonals ($r_{ij} = 0$ for $j > i+2$), and

$$A = QR, \qquad Q = G^{(1)\mathsf{T}} G^{(2)\mathsf{T}} \cdots G^{(n-1)\mathsf{T}}.$$

The total cost is $O(n)$.

```
def qr_tridiagonal(T):
    """QR factorization of a tridiagonal matrix with n - 1 Givens rotations."""
    R = T.astype(float).copy()
    n = R.shape[0]
    Q = np.eye(n)
    for k in range(n - 1):
        c, s = givens(R[k, k], R[k+1, k])
        G = np.array([[c, s], [-s, c]])
        cols = slice(k, min(k + 3, n))       # only three nonzero columns
        R[k:k+2, cols] = G @ R[k:k+2, cols]
        Q[:, k:k+2] = Q[:, k:k+2] @ G.T
    return Q, R
```

(Accumulating Q explicitly, as here for clarity, costs $O(n^2)$. In the QR algorithm one instead applies the rotations directly to form RQ, keeping the cost of a step at $O(n)$.)

Example 10.2. For

$$A = \begin{pmatrix} 3 & 1 & 0 \\ 1 & 3 & 1 \\ 0 & 1 & 3 \end{pmatrix},$$

the first rotation has $c_1 = 3/\sqrt{10}$, $s_1 = 1/\sqrt{10}$, and

$$G^{(1)} A = \begin{pmatrix} \sqrt{10} & 3\sqrt{10}/5 & 1/\sqrt{10} \\ 0 & 4\sqrt{10}/5 & 3/\sqrt{10} \\ 0 & 1 & 3 \end{pmatrix}.$$

The second acts on rows 2 and 3 with $c_2 = 4\sqrt{10}/\sqrt{185}$ and $s_2 = \sqrt{185}/37$, giving

$$R = \begin{pmatrix} \sqrt{10} & 3\sqrt{10}/5 & 1/\sqrt{10} \\ 0 & \sqrt{185}/5 & 27/\sqrt{185} \\ 0 & 0 & 21/\sqrt{74} \end{pmatrix}, \qquad Q = \begin{pmatrix} 3/\sqrt{10} & -4/\sqrt{185} & 1/\sqrt{74} \\ 1/\sqrt{10} & 12/\sqrt{185} & -3/\sqrt{74} \\ 0 & \sqrt{185}/37 & 8/\sqrt{74} \end{pmatrix}.$$

The function `qr_tridiagonal` reproduces these entries, and $QR = A$ to machine precision.

Cost of reduction by Givens rotations. Reducing a full symmetric matrix to tridiagonal form with Givens rotations takes $(n-1)(n-2)/2$ rotations. Eliminating the entries of column k costs about $4(n-k)^2$ multiplications, for a total of $\sum_k 4(n-k)^2 \approx \frac{4}{3}n^3$ multiplications, about twice the Householder cost. This is why Householder reflectors are used for the reduction and Givens rotations for the QR steps on the reduced matrix.

10.3 How fast it converges, and when it does not

Let A be upper Hessenberg with eigenvalues $|\lambda_1| \geq |\lambda_2| \geq ... \geq |\lambda_n|$.

1. If $|\lambda_p| > |\lambda_{p+1}|$, the subdiagonal entry $a^{(k)}_{p+1,p}$ tends to zero linearly:

$$|a^{(k)}_{p+1,p}| = O\big(|\lambda_{p+1}/\lambda_p|^k\big)\,.$$

2. If all eigenvalues are real with distinct moduli, $A^{(k)}$ converges to an upper triangular matrix with $\lambda_1, \dots, \lambda_n$ on the diagonal (diagonal, if A is symmetric).

3. If $|\lambda_p| = |\lambda_{p+1}|$, for example $\lambda_{p+1} = -\lambda_p$, the factor equals one and $a^{(k)}_{p+1,p}$ need not tend to zero, so $A^{(k)}$ need not become triangular.

4. If λ_n is much closer to zero than all the other eigenvalues, $a^{(k)}_{n,n-1}$ tends to zero very fast. Shifts (Section 12) create this situation on purpose.

5. *Deflation.* When some $|a^{(k)}_{j+1,j}|$ falls below machine precision times the size of the neighboring diagonal entries, it is set to zero and $A^{(k)}$ splits into two independent problems,

$$A^{(k)} = \begin{pmatrix} A_1^{(k)} & * \\ 0 & A_2^{(k)} \end{pmatrix}, \qquad \sigma(A) = \sigma(A_1^{(k)}) \cup \sigma(A_2^{(k)}),$$

   by Lemma 7.1. The QR algorithm then continues on each block separately, possibly in parallel.

Example 10.3. For the 3×3 tridiagonal matrix of the previous example, with eigenvalues $3+\sqrt{2} \approx 4.4142$, 3 and $3-\sqrt{2} \approx 1.5858$, the predicted factors are $3/4.4142 \approx 0.68$ for a_{21} and $1.5858/3 \approx 0.53$ for a_{32}. The unshifted QR iteration gives

k	$\lvert a^{(k)}_{21}\rvert$	$\lvert a^{(k)}_{32}\rvert$
1	0.860	0.897
5	0.279	0.0840
10	0.0420	0.00341

Examples where the basic algorithm fails.

1. $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$, with eigenvalues ± 1. Here $A = QR$ with $Q = A$ and $R = I$, so $A^{(1)} = RQ = A$, and the iteration never moves.

2. The 3×3 permutation matrix with ones on the antidiagonal has eigenvalues $1, 1, -1$. A Hessenberg form of it is $\begin{pmatrix} 0 & -1 & 0 \\ -1 & 0 & 0 \\ 0 & 0 & 1 \end{pmatrix}$, which again contains a block with eigenvalues ± 1, and the QR algorithm leaves the matrix unchanged. In general, a symmetric matrix with two eigenvalues $\lambda_p = -\lambda_{p+1}$ does not converge to diagonal form.

3. The companion matrix C of $x^5 + 1$ has the five fifth roots of -1 as eigenvalues, all of modulus one. Its QR factorization is $C = (-C)(-I)$, so $A^{(1)} = (-I)(-C) = C$, and again nothing happens.

In each case the eigenvalues have equal moduli, so the ratios that drive convergence equal one. The cure is to apply the algorithm to $A - \alpha I$, which changes the moduli to $|\lambda_i - \alpha|$ and breaks the ties (Section 12).

11 Simultaneous iteration and why QR works

11.1 Gram–Schmidt is QR

Recall that Gram–Schmidt orthogonalization of linearly independent vectors $v_1, \dots, v_m$ produces orthonormal $q_1, \dots, q_m$ with $\operatorname{span}(q_1, \dots, q_j) = \operatorname{span}(v_1, \dots, v_j)$ for every j. In matrix form this is the reduced QR factorization $[v_1, \dots, v_m] = [q_1, \dots, q_m]R$ with R upper triangular. Classical Gram–Schmidt is numerically unstable; in practice one uses modified Gram–Schmidt or Householder QR.

11.2 Simultaneous power iteration

Let A be real symmetric with orthonormal eigenvectors $q_1, \dots, q_n$ and eigenvalues

$$|\lambda_1| > |\lambda_2| > \cdots > |\lambda_m| > |\lambda_{m+1}| \geq \dots \geq |\lambda_n|.$$

Start the power method from m vectors at once, $V^{(0)} = [v_1^{(0)}, \dots, v_m^{(0)}]$, and form $V^{(k)} = A^k V^{(0)}$. Let $\hat{Q} = [q_1, \dots, q_m]$ and assume that all leading principal minors of $\hat{Q}^{\mathsf{T}} V^{(0)}$ are nonzero.

Theorem 11.1. *Let $V^{(k)} = \hat{Q}^{(k)} R^{(k)}$ be the reduced QR factorization of $A^k V^{(0)}$. Then the columns of $\hat{Q}^{(k)}$ converge to $\pm q_1, \dots, \pm q_m$, with*

$$\| q_j^{(k)} - (\pm q_j) \| = O(C^k), \qquad C = \max_{1 \leq j \leq m} \frac{|\lambda_{j+1}|}{|\lambda_j|}.$$

Sketch. Write $V^{(k)} = Q \Lambda^k Q^{\mathsf{T}} V^{(0)} = \hat{Q} \hat{\Lambda}^k \hat{Q}^{\mathsf{T}} V^{(0)} + O(|\lambda_{m+1}|^k)$, where $\hat{\Lambda} = \operatorname{diag}(\lambda_1, \dots, \lambda_m)$. Since $\hat{Q}^{\mathsf{T}} V^{(0)}$ is nonsingular, $V^{(k)} = \big(\hat{Q} \hat{\Lambda}^k + O(|\lambda_{m+1}|^k) \big) \hat{Q}^{\mathsf{T}} V^{(0)}$, so the column space of $V^{(k)}$ is close to that of $\hat{Q}$. Because every leading principal minor is nonzero, the same argument applies to the first j columns for each j: the span of the first j columns of $V^{(k)}$ approaches $\operatorname{span}(q_1, \dots, q_j)$, at a rate governed by $|\lambda_{j+1}/\lambda_j|$. QR factorization preserves these nested spans, so $q_j^{(k)} \to \pm q_j$. $\square$

Computed literally, this is useless: every column of $A^k V^{(0)}$ converges to the direction of q_1, so $V^{(k)}$ becomes extremely ill-conditioned and the information about the other eigenvectors is lost to rounding. The stable version re-orthogonalizes at every step:

1. Choose $\widehat{Q}^{(0)} \in \mathbb{R}^{n\times m}$ with orthonormal columns.
2. For $k = 1, 2, \ldots$: $Z^{(k)} = A\widehat{Q}^{(k-1)}$, and factor $Z^{(k)} = \widehat{Q}^{(k)}\widehat{R}^{(k)}$.

By induction, $Z^{(k)}\widehat{R}^{(k-1)} \cdots \widehat{R}^{(1)} = A^k \widehat{Q}^{(0)} = V^{(k)}$, so $\widehat{Q}^{(k)}$ has the same nested column spaces as $V^{(k)}$, and Theorem 11.1 applies unchanged: $q_j^{(k)} \to \pm q_j$.

11.3 Simultaneous iteration is the QR algorithm

Take $m = n$ and $\widehat{Q}^{(0)} = I$, and compare two algorithms. In both, write $\underline{Q}^{(k)}$ for the accumulated orthogonal matrix and $\underline{R}^{(k)} = R^{(k)}R^{(k-1)}\cdots R^{(1)}$, and assume all QR factorizations are normalized so that R has positive diagonal entries (this makes them unique).

Simultaneous iteration	Unshifted QR algorithm
$\underline{Q}^{(0)} = I$	$A^{(0)} = A$
$Z = A\underline{Q}^{(k-1)}$	$A^{(k-1)} = Q^{(k)}R^{(k)}$
$Z = \underline{Q}^{(k)}R^{(k)}$	$A^{(k)} = R^{(k)}Q^{(k)}$
$A^{(k)} = \underline{Q}^{(k)\mathsf{T}} A \underline{Q}^{(k)}$	$\underline{Q}^{(k)} = Q^{(1)}Q^{(2)}\cdots Q^{(k)}$

Theorem 11.2. *The two algorithms generate identical matrices $\underline{Q}^{(k)}$, $\underline{R}^{(k)}$ and $A^{(k)}$, and*

$$A^k = \underline{Q}^{(k)}\underline{R}^{(k)}, \qquad A^{(k)} = \underline{Q}^{(k)\mathsf{T}} A\, \underline{Q}^{(k)}.$$

Proof. By induction on k; the case $k = 0$ is trivial. *Simultaneous iteration:* the second formula holds by definition, and if $A^{k-1} = \underline{Q}^{(k-1)}\underline{R}^{(k-1)}$ then

$$A^k = A\underline{Q}^{(k-1)}\underline{R}^{(k-1)} = Z\underline{R}^{(k-1)} = \underline{Q}^{(k)}R^{(k)}\underline{R}^{(k-1)} = \underline{Q}^{(k)}\underline{R}^{(k)}.$$

QR algorithm: assume $A^{k-1} = \underline{Q}^{(k-1)}\underline{R}^{(k-1)}$ and $A^{(k-1)} = \underline{Q}^{(k-1)\mathsf{T}}A\underline{Q}^{(k-1)}$, that is, $A\underline{Q}^{(k-1)} = \underline{Q}^{(k-1)}A^{(k-1)}$. Then

$$A^k = A\underline{Q}^{(k-1)}\underline{R}^{(k-1)} = \underline{Q}^{(k-1)}A^{(k-1)}\underline{R}^{(k-1)} = \underline{Q}^{(k-1)}Q^{(k)}R^{(k)}\underline{R}^{(k-1)} = \underline{Q}^{(k)}\underline{R}^{(k)},$$

and $A^{(k)} = Q^{(k)\mathsf{T}}A^{(k-1)}Q^{(k)} = Q^{(k)\mathsf{T}}\cdots Q^{(1)\mathsf{T}}AQ^{(1)}\cdots Q^{(k)} = \underline{Q}^{(k)\mathsf{T}}A\underline{Q}^{(k)}$. Since A^k has a unique QR factorization with positive diagonal in R, the two algorithms produce the same $\underline{Q}^{(k)}$ and $\underline{R}^{(k)}$, hence the same $A^{(k)}$. □

So the QR algorithm is simultaneous power iteration on the full identity matrix, written in a form that tracks $A^{(k)}$ directly. Combining this with Theorem 11.1 gives the convergence result.

Theorem 11.3. *Let the unshifted QR algorithm be applied to a real symmetric A with $|\lambda_1| > |\lambda_2| > \cdots > |\lambda_n|$, whose eigenvector matrix Q has nonzero leading principal minors. Then $A^{(k)} \to \operatorname{diag}(\lambda_1, \ldots, \lambda_n)$ and $\underline{Q}^{(k)} \to Q$ (with signs adjusted as necessary), both linearly with constant $C = \max_j |\lambda_{j+1}|/|\lambda_j|$.*

Remark 11.4. NumPy's `numpy.linalg.qr` does not normalize the signs of the diagonal of R, so in a numerical experiment the two algorithms agree only up to the signs of the columns of $\underline{Q}^{(k)}$; the diagonal of $A^{(k)}$ agrees exactly. For the 3×3 example above, ten steps of either method give diagonal entries 4.412965, 3.001241, 1.585795.

The same equivalence explains the role of the first and last columns. The first column of $\underline{Q}^{(k)}$ is the normalized $A^k e_1$, the power method started at e_1. Inverting $A^k = \underline{Q}^{(k)} \underline{R}^{(k)}$ and using symmetry gives $A^{-k} = \underline{Q}^{(k)} (\underline{R}^{(k)})^{-\mathsf{T}}$, with $(\underline{R}^{(k)})^{-\mathsf{T}}$ lower triangular, so the *last* column of $\underline{Q}^{(k)}$ is the normalized $A^{-k} e_n$, the inverse power method started at e_n. This is why the bottom-right corner converges to the smallest eigenvalue and why shifts are applied there.

12 The QR algorithm with shifts

12.1 The algorithm

The last column of $\underline{Q}^{(k)}$ performs inverse iteration, and inverse iteration is fast when it is shifted close to an eigenvalue. The practical algorithm for a symmetric matrix is:

1. Reduce A to tridiagonal form $A^{(0)}$ (Section 9).
2. For $k = 1, 2, \ldots$:
   (a) choose a shift $\alpha^{(k)}$;
   (b) factor $A^{(k-1)} - \alpha^{(k)} I = Q^{(k)} R^{(k)}$;
   (c) set $A^{(k)} = R^{(k)} Q^{(k)} + \alpha^{(k)} I$;
   (d) if some off-diagonal entry $a^{(k)}_{j+1,j}$ is negligible, set it to zero, split $A^{(k)} = \operatorname{diag}(A_1, A_2)$, and apply the algorithm to A_1 and A_2 separately.

Each step is still a similarity transformation, $A^{(k)} = Q^{(k)\mathsf{T}} A^{(k-1)} Q^{(k)}$, because the shift is subtracted and added back. If α is close to an eigenvalue, the subdiagonal entry converges with the factor $|\lambda_{p+1} - \alpha|/|\lambda_p - \alpha|$, and the entry $a^{(k)}_{n,n-1}$ converges fastest when α is closer to some λ than to any other eigenvalue. With $\underline{Q}^{(k)} = Q^{(1)} \cdots Q^{(k)}$ and $\underline{R}^{(k)} = R^{(k)} \cdots R^{(1)}$, the shifted algorithm satisfies

$$(A - \alpha^{(k)} I) \cdots (A - \alpha^{(1)} I) = \underline{Q}^{(k)} \underline{R}^{(k)},$$

so it performs simultaneous iteration with a polynomial in A instead of a power of A.

12.2 Choosing the shift

Rayleigh quotient shift. Take $\alpha^{(k)} = a^{(k-1)}_{nn}$, the current bottom-right entry. Since $A^{(k-1)} = \underline{Q}^{(k-1)\mathsf{T}} A \underline{Q}^{(k-1)}$, this number equals $q_n^{\mathsf{T}} A q_n$, where q_n is the last column of $\underline{Q}^{(k-1)}$: it is the Rayleigh quotient of the vector produced by inverse iteration.

Remark 12.1. Some texts list the "Rayleigh shift" a_{nn} and the "Rayleigh quotient shift" $q_n^{\mathsf{T}} A q_n$ as two strategies. By the identity just stated they are the same number. With this shift the last column of $\underline{Q}^{(k)}$ and the shift itself coincide with the iterates of Rayleigh quotient iteration started at $x^{(0)} = e_n$. Consequently convergence is cubic for symmetric matrices in the generic case (quadratic for nonsymmetric matrices).

The Rayleigh quotient shift can fail. For $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ the shift is $a_{22} = 0$, so $A - 0I = A$ and the iteration is stuck exactly as before: the shift sits exactly halfway between the eigenvalues ± 1 and favors neither.

Wilkinson shift. Take the eigenvalue of the trailing 2×2 block

$$\begin{pmatrix} a_{m-1} & b_m \\ b_m & a_m \end{pmatrix}$$

that is closer to a_m. In a numerically stable form,

$$\mu = a_m - \frac{\operatorname{sign}(\delta)\, b_m^2}{|\delta| + \sqrt{\delta^2 + b_m^2}}, \qquad \delta = \frac{a_{m-1} - a_m}{2}, \qquad \operatorname{sign}(0) = 1.$$

For symmetric tridiagonal matrices the QR algorithm with the Wilkinson shift always converges, cubically in the generic case and at least quadratically in the worst case (Trefethen and Bau, Lecture 29). It is the shift used in practice.

Example 12.2. For $A = \begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ the Wilkinson shift is an exact eigenvalue, $\delta = 0$, $\mu = 0 - 1 = -1$. Then

$$A + I = \begin{pmatrix} 1 & 1 \\ 1 & 1 \end{pmatrix} = QR, \quad Q = \frac{1}{\sqrt{2}} \begin{pmatrix} -1 & -1 \\ -1 & 1 \end{pmatrix}, \quad R = \begin{pmatrix} -\sqrt{2} & -\sqrt{2} \\ 0 & 0 \end{pmatrix},$$

$$RQ = \begin{pmatrix} 2 & 0 \\ 0 & 0 \end{pmatrix}, \qquad A^{(1)} = RQ - I = \begin{pmatrix} 1 & 0 \\ 0 & -1 \end{pmatrix}.$$

The shifted algorithm converges in a single step, where the unshifted one never moves.

12.3 Deflation in practice

With the Wilkinson shift the last off-diagonal entry b_m goes to zero, typically in two or three iterations. Once $|b_m|$ is negligible, a_m is accepted as an eigenvalue and the algorithm continues on the leading $(m-1) \times (m-1)$ block:

1. Set $A^{(0)}$ to the tridiagonal form of A and $m = n$.
2. While $m \geq 2$: repeat Wilkinson-shifted QR steps on the leading $m \times m$ block until $|b_m| < \text{tol}$; record a_m as an eigenvalue; set $m \leftarrow m - 1$.
3. Record a_1 as the last eigenvalue.

```
def wilkinson_shift(a_prev, a_last, b):
    """Eigenvalue of [[a_prev, b], [b, a_last]] closest to a_last."""
    delta = (a_prev - a_last) / 2.0
    sgn = 1.0 if delta >= 0 else -1.0
    return a_last - sgn * b**2 / (abs(delta) + np.hypot(delta, b))

def tridiagonal_qr_eig(T, tol=1e-14, maxit=100):
    """Eigenvalues of a symmetric tridiagonal T: shifted QR with deflation."""
    T = T.astype(float).copy()
    eigs = []
    iters = []
    m = T.shape[0]
    while m > 1:
        for k in range(maxit):
            if abs(T[m-1, m-2]) < tol * (abs(T[m-1, m-1]) + abs(T[m-2, m-2])):
                break
            mu = wilkinson_shift(T[m-2, m-2], T[m-1, m-1], T[m-1, m-2])
            Q, R = qr_tridiagonal(T[:m, :m] - mu * np.eye(m))
            T[:m, :m] = R @ Q + mu * np.eye(m)
        else:
            raise RuntimeError("no convergence")
        eigs.append(T[m-1, m-1])
        iters.append(k)
        m -= 1                                  # deflate: drop last row/column
    eigs.append(T[0, 0])
    return np.sort(np.array(eigs)), iters
```

Example 12.3. A test on a random symmetric 6×6 matrix:

```
rng = np.random.default_rng(0)
M = rng.standard_normal((6, 6))
S = M + M.T
T6, _ = hessenberg_reduce(S)
ev, its = tridiagonal_qr_eig(T6)
print(ev)
print(np.linalg.eigvalsh(S))
print(np.max(np.abs(ev - np.linalg.eigvalsh(S))), its)
```

prints

```
[-4.007424 -1.975998  0.552206  0.982674  3.000282  3.63028 ]
[-4.007424 -1.975998  0.552206  0.982674  3.000282  3.63028 ]
7.105427357601002e-15 [3, 2, 4, 1, 1]
```

(with `np.set_printoptions(precision=6)`). The eigenvalues agree with LAPACK to about 7×10^{-15}, and the five deflations took $3, 2, 4, 1, 1$ shifted QR steps: thirteen steps for six eigenvalues.

12.4 Computing the eigenvectors

Suppose the reduction gave $A^{(0)} = HAH^{\mathsf{T}}$ with H orthogonal, so $AH^{\mathsf{T}} = H^{\mathsf{T}}A^{(0)}$. If $A^{(0)}w = \lambda w$, then

$$A(H^{\mathsf{T}}w) = H^{\mathsf{T}}A^{(0)}w = \lambda\,(H^{\mathsf{T}}w),$$

so $(\lambda, H^{\mathsf{T}}w)$ is an eigenpair of A. The QR iteration gives $A^{(k)} = \underline{Q}^{(k)\mathsf{T}}A^{(0)}\underline{Q}^{(k)}$ with $\underline{Q}^{(k)} = Q^{(1)}\cdots Q^{(k)}$. For symmetric A, $A^{(k)}$ tends to a diagonal matrix, so $A^{(0)}\underline{Q}^{(k)} \approx \underline{Q}^{(k)}\Lambda$: the columns $q_1, \dots, q_n$ of the limit of $\underline{Q}^{(k)}$ are eigenvectors of $A^{(0)}$. The eigenvectors of A are therefore

$$v_i = H^{\mathsf{T}}q_i, \qquad i = 1, \dots, n.$$

When the algorithm deflates, an $m \times m$ factor $Q^{(k)}$ is embedded as $\operatorname{diag}(Q^{(k)}, I_{n-m})$ before accumulating. Accumulating $\underline{Q}$ costs $O(n^3)$ flops in total, far more than the $O(n^2)$ needed for the eigenvalues of a tridiagonal matrix alone; when only a few eigenvectors are wanted, it is cheaper to run inverse iteration with the computed eigenvalues as shifts.

12.5 Summary

Method	Finds	Convergence
Power method	λ of largest modulus	Linear, factor $\lvert\lambda_2/\lambda_1\rvert$
Symmetric power method	λ of largest modulus	Linear, factor $\lvert\lambda_2/\lambda_1\rvert^2$
Shifted inverse power	λ closest to α	Linear, factor $\lvert(\lambda_1-\alpha)/(\lambda_2-\alpha)\rvert$
Rayleigh quotient iteration	One eigenpair	Cubic (symmetric)
Wielandt deflation	Next eigenvalue	As the power method
Unshifted QR	All eigenvalues	Linear, factors $\lvert\lambda_{j+1}/\lambda_j\rvert$
QR with Wilkinson shifts	All eigenvalues	Cubic (generic, symmetric)

The algorithm used by `numpy.linalg.eigh` and `eigvalsh` (LAPACK) for a dense symmetric matrix is: reduce to tridiagonal form in about $\frac{4}{3}n^3$ flops, then run shifted QR (or a related divide-and-conquer method) on the tridiagonal matrix, which costs only $O(n^2)$ for all the eigenvalues. For nonsymmetric matrices `numpy.linalg.eig` reduces to Hessenberg form and runs a double-shift QR algorithm in real arithmetic, which converges to the real Schur form.

13 Practice problems

Problem 1. Apply two steps of the power method with infinity-norm scaling to $A = \begin{pmatrix} 2 & 1 \\ 1 & 2 \end{pmatrix}$, starting from $x^{(0)} = (1, 0)^{\mathsf{T}}$. What do $\beta^{(k)}$ and $x^{(k)}$ converge to, and with what factor?

Problem 2. The eigenvalues of a matrix are 10, 9.5, 3 and 1. How many power method iterations are needed to reduce the error by a factor 10^{-6}? With what shift would the shifted inverse power method find $\lambda = 10$ with a convergence factor of 0.1 or less?

Problem 3. Show that if A is symmetric and $x = v + \varepsilon w$ with $Av = \lambda v$, $\|v\|_2 = \|w\|_2 = 1$ and $w \perp v$, then $r(x) - \lambda = O(\varepsilon^2)$.

Problem 4. Find the Householder vector u and α that map $w = (3,4)^{\mathsf{T}}$ to αe_1, and verify $(I - 2uu^{\mathsf{T}})w = \alpha e_1$.

Problem 5. Show that one step of the QR algorithm preserves the upper Hessenberg form.

Problem 6. Compute the Wilkinson shift for the trailing block $\begin{pmatrix} 2 & 1 \\ 1 & 3 \end{pmatrix}$, and verify that it is an eigenvalue of that block.

Problem 7. Explain why the unshifted QR algorithm fails for $\begin{pmatrix} 0 & 1 \\ 1 & 0 \end{pmatrix}$ and why the shift $\alpha = a_{nn}$ does not help, but the Wilkinson shift does.

Solutions

Problem 1. $Ax^{(0)} = (2,1)^{\mathsf{T}}$, $\beta^{(1)} = 2$, $x^{(1)} = (1, \tfrac{1}{2})^{\mathsf{T}}$; $Ax^{(1)} = (\tfrac{5}{2}, 2)^{\mathsf{T}}$, $\beta^{(2)} = \tfrac{5}{2}$, $x^{(2)} = (1, \tfrac{4}{5})^{\mathsf{T}}$. The eigenvalues are 3 and 1 with eigenvectors $(1,1)^{\mathsf{T}}$ and $(1,-1)^{\mathsf{T}}$, so $\beta^{(k)} \to 3$ and $x^{(k)} \to (1,1)^{\mathsf{T}}$, with factor $1/3$.

Problem 2. The factor is 0.95, and $0.95^k \le 10^{-6}$ needs $k \ge \ln 10^{-6} / \ln 0.95 \approx 270$. With shift α near 10 the factor is $|10-\alpha|/|9.5-\alpha|$. Taking $\alpha > 10$, this is at most 0.1 when $\alpha - 10 \le 0.1(\alpha - 9.5)$, that is $\alpha \le 10.056$; for $\alpha < 10$ one needs $10 - \alpha \le 0.1(\alpha - 9.5)$, that is $\alpha \ge 9.955$. Any $\alpha \in [9.955, 10.056]$, $\alpha \ne 10$, works.

Problem 3. $x^{\mathsf{T}}Ax = \lambda + \varepsilon^2 w^{\mathsf{T}}Aw$ (the cross terms vanish because $w^{\mathsf{T}}Av = \lambda w^{\mathsf{T}}v = 0$), and $x^{\mathsf{T}}x = 1 + \varepsilon^2$. So $r(x) - \lambda = \varepsilon^2(w^{\mathsf{T}}Aw - \lambda)/(1+\varepsilon^2) = O(\varepsilon^2)$.

Problem 4. $\|w\|_2 = 5$, $\alpha = -5$, $y = w - \alpha e_1 = (8,4)^{\mathsf{T}}$, $u = (2,1)^{\mathsf{T}}/\sqrt{5}$. Then $u^{\mathsf{T}}w = 10/\sqrt{5}$ and $w - 2u(u^{\mathsf{T}}w) = (3,4)^{\mathsf{T}} - 4(2,1)^{\mathsf{T}} = (-5,0)^{\mathsf{T}} = \alpha e_1$.

Problem 5. If $A^{(k-1)} = QR$ is Hessenberg and nonsingular, then $Q = A^{(k-1)}R^{-1}$ is Hessenberg (a Hessenberg matrix times an upper triangular one). Then RQ is an upper triangular matrix times a Hessenberg one, which is again Hessenberg. (The singular case follows by a continuity argument, or directly from the Givens construction.)

Problem 6. $\delta = (2-3)/2 = -\tfrac{1}{2}$, so $\operatorname{sign}(\delta) = -1$ and $\mu = 3 + 1/(\tfrac{1}{2} + \sqrt{\tfrac{1}{4} + 1}) = 3 + 2/(1+\sqrt{5}) = (5+\sqrt{5})/2 \approx 3.618$. The block has characteristic polynomial $\lambda^2 - 5\lambda + 5$ with roots $(5 \pm \sqrt{5})/2$; μ is the root closer to $a_m = 3$.

Problem 7. The eigenvalues ± 1 have equal modulus, so the convergence factor is one; concretely $A = A \cdot I$ is a QR factorization and $RQ = A$. The shift $a_{22} = 0$ leaves the matrix unchanged. The Wilkinson shift is -1, an exact eigenvalue, and one step gives $\operatorname{diag}(1,-1)$.

References

G. H. Golub and C. F. Van Loan, *Matrix Computations*, 4th ed., Johns Hopkins University Press, 2013.

D. Kincaid and W. Cheney, *Numerical Analysis*, Brooks/Cole, 2002.

J. Stoer and R. Bulirsch, *Introduction to Numerical Analysis*, Springer, 2002.

L. N. Trefethen and D. Bau, III, *Numerical Linear Algebra*, SIAM, 1997, Lectures 24–30.